

Mini Project 1

Oleksandr Shepeliuk and Naga Pulligilla

1. Introduction:

My partner and I were tasked with creating a program that can use both sorted and unsorted lists to handle simulated phone call records stored in text files. Each line represents a call record and includes the following data: source phone number, target phone number, date the call started (MM/DD/YYYY), call started (HH:MM:SS), duration of the call (in seconds). These phone records belong to an imaginary country, where phone numbers have 9 digits in the following format: (AA)-BB-CCCCC, where AA is the two-digit region code, BB is the two-digit subregion code, and CCCCC is the local phone number.

With this in mind, we built a program that reads the records from the files and stores them. This is done using two indexing strategies: Sorted and Unsorted. The list implementation is done either with an ArrayList or a LinkedList. Therefore, there are four possible configurations. We have worked with creating a few different classes for this project.

2. Code architecture:

My classmate and I created a program that reads CSV phone-call records, creates Call objects, stores them in either UnsortedCallDB or SortedCallDB, and allows searching by source phone number.

Each class in our program has its own function and completes the whole.

- Call - stores one call: source, target, date, time duration.
- PhoneCalls - stores all calls from one source number
- UnsortedCallDB - stores all calls in one unsorted list and searches sequentially.
- SortedCallDB - stores calls grouped by source number in sorted order and searches using binary search.
- Main - handles user input, file reading, indexing, searching, and benchmarking.

CSV File

|

v

Main.readFile()

|

v

Call objects

|

+--> UnsortedCallDB

| |

| v

| List<Call>

|

+--> SortedCallDB

|

v

List<PhoneCalls>

|

v

each PhoneCalls stores List<Call>

For UnsortedCallDB :

index_call(call) - add call to end of list

search_call(number) - loop through list and collect matching calls

For SortedCallsDB:

index_calls(call) - find correct source-number position and insert/groupcall

search_calls(number) - binary search source number

We also created an automated mode which runs a chosen method and a chosen type of list on a file provided by the user 5 times, records the time, sorts the values (if needed), and then computes the minimum, maximum, mean, and median. We modified the data collection process because SortedCallDB, specifically the LinkedList implementation, is a very slow and inefficient way to complete this task, as you will see from the data below. That's why we implemented a user-selectable automated mode. Therefore, completing the entire data collection may take a few days.

3. Computer Specs:

Due to time constraints, we split our data collection evenly. Oleksandr collected data on Indexing, while I (Naga) ran tests on search results.

Oleksandr:

Processor: AMD Ryzen 5 7530U with Radeon Graphics (2.00 GHz)

Memory: 16 GB RAM (15.3 GB usable)

Graphics: AMD Radeon Graphics

Storage: 477 GB SSD

Operating System: Windows 11 Home (64-bit)

Naga:

Model: MacBook Air (MacBookAir10,1)

Chip: Apple M1

Cores: 8 (4 performance, 4 efficiency)

Memory: 8 GB

OS: macOS (System Firmware 8422.141.2)

4. Results Part 1. UnsortedCallDB.

The UnsortedCallDB implementation proved to be the most efficient configuration in this project. Both the ArrayList and LinkedList versions showed fast indexing and search performance across all file sizes. Based on the collected data, indexing the largest dataset containing 256,000 call records required less than one second, demonstrating excellent efficiency.

Search operations get slower as the file grows. It is not sorted, so to find a specific number, the program must iterate through the entire list of calls. In the worst case, it may require $O(N)$ iterations, which makes searching slower for larger files.

For unsorted method indexing has been working with almost the same speed for ArrayList and LinkedList. The collected data shows only minor differences in execution time across all tested file sizes. Since this is unsorted method, the program just adds a new call to the end of a list. So, we can say that unsorted ways of indexing information is efficient for both types of lists.

Index	List	Size	Min	Max	Mean	Median
Unsorted	ArrayList	4000	0.0067	0.0323	0.0141	0.0110
Unsorted	ArrayList	16000	0.0125	0.0631	0.0288	0.0254
Unsorted	ArrayList	64000	0.0317	0.1327	0.0587	0.0410

Unsorted	ArrayList	256000	0.1221	0.2679	0.1679	0.1573
Unsorted	LinkedList	4000	0.0076	0.0291	0.0144	0.0111
Unsorted	LinkedList	16000	0.0147	0.0663	0.0326	0.0300
Unsorted	LinkedList	64000	0.0480	0.1312	0.0688	0.0548
Unsorted	LinkedList	256000	0.1594	0.2894	0.2062	0.1845

Index	List	Size	Existing Mean	Existin g Media n	Non- existing Mean	Non- existin g Media n	All Mean	All Median
Unsorte d	ArrayList	4000	0.00091 2	0.0013 7	0.00050 4	0.0003 4	0.00070 8	0.00059 5
Unsorte d	ArrayList	16000	0.00209 8	0.0018 5	0.00111 8	0.0009 7	0.00160 8	0.00122 5
Unsorte d	ArrayList	64000	0.00315 8	0.0033 1	0.00347 2	0.0034 1	0.00331 5	0.00323 5
Unsorte d	ArrayList	25600 0	0.01100 4	0.0065 7	0.00912 4	0.0115 7	0.01006 4	0.00836 5
Unsorte d	LinkedLi st	4000	0.00169 4	0.0016 5	0.00090 0	0.0008 1	0.00129 7	0.00109 5
Unsorte d	LinkedLi st	16000	0.00421 4	0.0042 3	0.00457 6	0.0039 6	0.00439 5	0.00409 5

Unsorted	LinkedList	64000	0.006480	0.00591	0.006998	0.00639	0.006739	0.006150
Unsorted	LinkedList	256000	0.022884	0.02280	0.023036	0.02237	0.022960	0.022585

An in-depth analysis of these results.

Effect of List Type

For indexing, ArrayList and LinkedList performed very similarly. The differences in median indexing time were small across all file sizes. This is expected because the UnsortedCallDB implementation simply appends new calls to the end of the list without searching for an insertion position.

For searching, ArrayList consistently outperformed LinkedList. For example, with 256,000 calls, the median search time was approximately 0.008365 seconds for ArrayList and 0.022585 seconds for LinkedList. This difference is likely caused by the faster element access provided by ArrayList.

Effect of File Size

Indexing time increased as file size increased. However, the increase was gradual and remained below one second even for the largest dataset containing 256,000 calls.

Search time also increased as file size increased. Since UnsortedCallDB performs a sequential search, larger datasets require more comparisons before the desired phone number is found.

Interesting Observations

The difference between existing and non-existing phone numbers was relatively small. This occurs because the program often needs to examine a large portion of the list regardless of whether the number exists.

The results demonstrate that UnsortedCallDB is highly efficient for indexing but less efficient for searching. This tradeoff makes it suitable for applications where data is inserted frequently but searched less often.

5. Results Part 2. SortedCallDB.

SortedCallDB works in a more organized way by keeping call records and ordering them by phone number. It is like a filing cabinet, where each drawer corresponds to a phone number, and each call under that number goes into the drawer where it belongs. The drawers are also sorted by the phone number. For **Indexing**, when a new call comes in, the program goes through the cabinet/ if a drawer exists for the call, it gets added to the drawer. If there is no drawer for the particular phone number, it creates a new drawer and is sorted. For **searching**, each search sees how the number compares to the middle drawer and goes left or right each time in a binary fashion until it finds the right drawer.

B. Results for Indexing and Searching:

Sorted	ArrayList	4000	0.0390	0.0963	0.0563	0.0474
Sorted	ArrayList	16000	0.9442	1.2892	1.1496	1.1662
Sorted	ArrayList	64000	14.7343	20.9623	18.5597	20.023
Sorted	ArrayList	256000	175.6952	226.8349	203.3039	204.4785
Sorted	LinkedList	4000	18.2920	18.8403	18.6050	18.6899
Sorted	LinkedList	16000	1438.888	2714.762	1898.431	1789.533
Sorted	LinkedList	64000*	25.6 hours	48.3 hours	33.7 hours	31.8 hours
Sorted	LinkedList	256000*	68.2 hours	128.7 hours	90.0 hours	84.8 hours

With LinkedList, the trial for 16k itself took hours. Given the timeframe we had to complete this project, we concluded it is best to approximate the remaining data, as it would take a very long time. Data for 64000 LinkedList and 256000 LinkedList are approximated. Values are estimated based on the observed growth between the 4K and 16K measurements. The actual benchmarks were not completed because the 16K configuration already required approximately 30 minutes median runtime.

SortedCallDB Search Summary:

Index	List	Size	Existing Mean	Existing Median	Non- existing Mean	Non- existing Median	All Mean	All Median
Sorted	ArrayList	4000	0.000022	0.00002	0.00002	0.00002	0.000021	0.00002
Sorted	ArrayList	16000	0.000028	0.00002	0.00002	0.00002	0.000024	0.00002
Sorted	ArrayList	64000	0.000016	0.00002	0.000022	0.00002	0.000019	0.00002
Sorted	ArrayList	256000	0.000018	0.00002	0.00002	0.00002	0.000019	0.00002
Sorted	LinkedList	4000	0.000778	0.00078	0.000768	0.00072	0.000773	0.000725
Sorted	LinkedList	16000	0.003534	0.00378	0.002894	0.00236	0.003214	0.003155

C. Analysis

Indexing:

ArrayList performed significantly better than LinkedList. This is mainly due to the sorted insertion, which repeatedly calls get(i). ArrayList accesses a position in constant time, so when the list grows, walking through the LinkedList can be very expensive. Each time the file grew, the indexing time grew around 16 times for ArrayList. This has the pattern of an

$O(N^2)$. For LinkedList, large files are practically unusable, as it would take huge amounts of time. In the end, we had to estimate the larger time as we simply did not have enough time. The LinkedList's `get(i)` method must walk from the beginning of the list every time it is run to a position, whereas ArrayList goes directly.

Search:

For ArrayList, search times were consistent and fast across all file sizes. Binary Search is very efficient. For LinkedList, the same Binary Search took a bit longer because it uses `get(mid)` repeatedly, and LinkedList's `get` method has to walk from the beginning of the list every single time, so the LinkedList search times were much slower. Search times for existing and non-existing numbers were quite similar, as Binary Search takes around the same number of steps whether or not the number exists in the list.

6. General Analysis.

When it comes to indexing, the unsorted option is faster. Unsorted categorizing just appends to the end. Of course, there is no sorting involved. SortedCallDB has to find the correct specific position for all calls, so it comes out a bit slower. This reflects on the numbers above. For example, UnsortedCallDB indexed a 256k file in about 0.2 seconds, while SortedCallDB took over 200 seconds. When it comes to searching, SortedCallDB is a wiser choice. SortedCallDB goes by binary search, so it is not as affected by the increasing file sizes either. UnsortedCallDB uses sequential search, so it becomes a problem if the files get to a big size. SortedCallDB search was around 0.00002 seconds with varying file sizes, while UnsortedCallDB jumped from 0.000595 for 4k to 0.008365 for 256k. For UnsortedCallDB, ArrayList and LinkedList can function very similarly. For SortedCallDB, ArrayList is significantly faster than LinkedList for both Indexing and Searching.

Index	List	Size	Min	Max	Mean	Median
Unsorted	ArrayList	4000	0.0067	0.0323	0.0141	0.0110
Unsorted	ArrayList	16000	0.0125	0.0631	0.0288	0.0254
Unsorted	ArrayList	64000	0.0317	0.1327	0.0587	0.0410

Unsorted	ArrayList	256000	0.1221	0.2679	0.1679	0.1573
Unsorted	LinkedList	4000	0.0076	0.0291	0.0144	0.0111
Unsorted	LinkedList	16000	0.0147	0.0663	0.0326	0.0300
Unsorted	LinkedList	64000	0.0480	0.1312	0.0688	0.0548
Unsorted	LinkedList	256000	0.1594	0.2894	0.2062	0.1845
Sorted	ArrayList	4000	0.0390	0.0963	0.0563	0.0474
Sorted	ArrayList	16000	0.9442	1.2892	1.1496	1.1662
Sorted	ArrayList	64000	14.7343	20.9623	18.5597	20.023
Sorted	ArrayList	256000	175.6952	226.8349	203.3039	204.4785
Sorted	LinkedList	4000	18.2920	18.8403	18.6050	18.6899
Sorted	LinkedList	16000	1438.888	2714.762	1898.431	1789.533
Sorted	LinkedList	64000*	25.6 hours	48.3 hours	33.7 hours	31.8 hours
Sorted	LinkedList	256000*	68.2 hours	128.7 hours	90.0 hours	84.8 hours

Sorted LinkedList 64000 and 256000 are approximated.

7. New application

Personally, one of my favorite things to do to relax is to watch movies. I like logging and keeping a record of what I watch throughout the year. A platform like this would store thousands of movies with their titles, crew, cast, year, etc. Users can build their personal lists with that data as well. My service would allow people to search by genre and maintain personal watchlists.

The system stores all movies in a catalog and will organize it by genre or sub-genre. Each Group holds a genre/sub-genre name, and a title contain any movie from that genre, like phone calls. The outer layer is a Genre group, which gets sorted by genre name alphabetically. From this, each user can make a personal list to make watchlists, their list of favorite movies, etc. This can be sorted as well separately.

The catalog needs a List type to store these movies per genre. The outer group is needed to organize by genre, similar to `List<PhoneCalls>`. There are many ways to sort these titles, so lists are necessary because the data will need to be dynamic. It cannot be an array as the size will be changing constantly. We need a type of data that handles growth automatically.

There are many filters that can be used for sorting. Beyond just the genre, users can sort by the release year, rating, box office, cast, etc. The structure can get a lot more diverse than the phone project. For example, within the genre group, instead of an unsorted list, we can maintain the list sorted by rating using the insertion from `SortedCallDB`. When a new movie is added to the genre, it gets inserted in the position based on its rating.

8. Conclusion

In this project, we implemented and analyzed two methods of indexing and searching data `UnsortedCallDB` and `SortedCallDB` using both `ArrayList` and `LinkedList`. The result of this project showed that `UnsortedCallDB` is the most efficient option for indexing, while `SortedCallDB` provides faster search performance through Binary Search.

The experiments also showed us that `ArrayList` generally performed better than `LinkedList`, especially in `SortedCallDB`. Overall, this project is a good example of how the choice of data structures and algorithms can significantly affect the performance of a program.

SEARCH RESULTS:**SortedCallDB + ArrayList + 4K:**

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00020	0.00005	0.00001	0.00011	0.00005	0.00005
(66)-05-07765	0.00016	0.00002	0.00001	0.00007	0.00002	0.00002
(60)-12-15378	0.00009	0.00001	0.00002	0.00009	0.00001	0.00002
(48)-16-73476	0.00014	0.00001	0.00001	0.00001	0.00013	0.00001
(38)-25-92205	0.00013	0.00001	0.00001	0.00001	0.00002	0.00001
(99)-99-99999	0.00013	0.00001	0.00002	0.00002	0.00005	0.00002
(88)-88-88888	0.00018	0.00002	0.00015	0.00002	0.00001	0.00002
(77)-77-77777	0.00016	0.00002	0.00002	0.00006	0.00001	0.00002
(66)-66-66666	0.00025	0.00002	0.00002	0.00002	0.00001	0.00002
(55)-55-55555	0.00015	0.00002	0.00001	0.00014	0.00002	0.00002

SortedCallDB + ArrayList + 16K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00020	0.00001	0.00003	0.00009	0.00006	0.00006
(66)-05-07765	0.00016	0.00002	0.00001	0.00002	0.00002	0.00002
(60)-12-15378	0.00017	0.00002	0.00002	0.00002	0.00006	0.00002

(48)-16-73476	0.00015	0.00007	0.00001	0.00002	0.00002	0.00002
(38)-25-92205	0.00017	0.00002	0.00002	0.00018	0.00002	0.00002
(99)-99-99999	0.00014	0.00002	0.00002	0.00002	0.00002	0.00002
(88)-88-88888	0.00016	0.00001	0.00002	0.00006	0.00002	0.00002
(77)-77-77777	0.00013	0.00002	0.00002	0.00001	0.00002	0.00002
(66)-66-66666	0.00016	0.00002	0.00002	0.00002	0.00041	0.00002
(55)-55-55555	0.00017	0.00001	0.00002	0.00002	0.00017	0.00002

SortedCallDB + ArrayList + 64k

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00015	0.00001	0.00001	0.00002	0.00004	0.00002
(66)-05-07765	0.00006	0.00002	0.00001	0.00002	0.00002	0.00002
(60)-12-15378	0.00018	0.00001	0.00001	0.00001	0.00001	0.00001

(48)-16-73476	0.00016	0.00001	0.00001	0.00001	0.00002	0.00001
(38)-25-92205	0.00008	0.00002	0.00001	0.00006	0.00001	0.00002
(99)-99-99999	0.00017	0.00002	0.00002	0.00002	0.00002	0.00002
(88)-88-88888	0.00017	0.00001	0.00002	0.00005	0.00002	0.00002
(77)-77-77777	0.00019	0.00002	0.00002	0.00002	0.00002	0.00002
(66)-66-66666	0.00017	0.00002	0.00003	0.00009	0.00002	0.00003
(55)-55-55555	0.00023	0.00002	0.00003	0.00002	0.00002	0.00002

SortedCallDB + ArrayList + 256K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00017	0.00001	0.00001	0.00006	0.00001	0.00001
(66)-05-07765	0.00010	0.00002	0.00002	0.00002	0.00001	0.00002
(60)-12-15378	0.00018	0.00001	0.00002	0.00002	0.00007	0.00002
(48)-16-73476	0.00016	0.00001	0.00002	0.00002	0.00006	0.00002
(38)-25-92205	0.00010	0.00002	0.00002	0.00002	0.00002	0.00002
(99)-99-99999	0.00155	0.00002	0.00002	0.00002	0.00002	0.00002
(88)-88-88888	0.00022	0.00002	0.00002	0.00013	0.00002	0.00002
(77)-77-77777	0.00097	0.00002	0.00002	0.00002	0.00002	0.00002

(66)-66-66666	0.00145	0.00002	0.00002	0.00002	0.00002	0.00002
---------------	---------	---------	---------	---------	---------	---------

(55)-55-55555	0.00164	0.00002	0.00002	0.00002	0.00002	0.00002
---------------	---------	---------	---------	---------	---------	---------

SortedCallDB + LinkedList + 4K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00263	0.00231	0.00059	0.00097	0.00059	0.00097
(66)-05-07765	0.00202	0.00067	0.00069	0.00052	0.00065	0.00067
(60)-12-15378	0.00191	0.00079	0.00086	0.00079	0.00110	0.00086
(48)-16-73476	0.00250	0.00068	0.00072	0.00090	0.00078	0.00078
(38)-25-92205	0.00204	0.00057	0.00059	0.00061	0.00124	0.00061
(99)-99-99999	0.00139	0.00064	0.00063	0.00071	0.00058	0.00064
(88)-88-88888	0.00218	0.00072	0.00056	0.00072	0.00079	0.00072
(77)-77-77777	0.00231	0.00056	0.00052	0.00071	0.00058	0.00058
(66)-66-66666	0.00210	0.00081	0.00071	0.00073	0.00051	0.00073
(55)-55-55555	0.00083	0.00084	0.00118	0.00117	0.00157	0.00117

SortedCallDB + LinkedList + 16K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00996	0.00225	0.00281	0.00231	0.00244	0.00244
(66)-05-07765	0.00734	0.00378	0.00316	0.00325	0.00479	0.00378
(60)-12-15378	0.00865	0.00449	0.00377	0.00385	0.00418	0.00418
(48)-16-73476	0.00693	0.00325	0.00407	0.00332	0.00518	0.00407
(38)-25-92205	0.00320	0.00500	0.00243	0.00439	0.00280	0.00320
(99)-99-99999	0.01291	0.00263	0.00233	0.00212	0.00220	0.00233
(88)-88-88888	0.00702	0.00236	0.00227	0.00261	0.00218	0.00236
(77)-77-77777	0.00522	0.00216	0.00195	0.00172	0.00225	0.00216
(66)-66-66666	0.00682	0.00307	0.00333	0.00234	0.00311	0.00311
(55)-55-55555	0.00918	0.00353	0.00409	0.00451	0.01024	0.00451

UnsortedCallDB + ArrayList + 4K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00154	0.00144	0.00153	0.00133	0.00126	0.00144
(66)-05-07765	0.00231	0.00121	0.00145	0.00148	0.00142	0.00145
(60)-12-15378	0.00233	0.00125	0.00159	0.00137	0.00095	0.00137
(48)-16-73476	0.00225	0.00067	0.00056	0.00056	0.00048	0.00056
(38)-25-92205	0.00213	0.00064	0.00057	0.00057	0.00065	0.00064
(99)-99-99999	0.00146	0.00109	0.00021	0.00023	0.00174	0.00109
(88)-88-88888	0.00171	0.00021	0.00060	0.00020	0.00076	0.00060
(77)-77-77777	0.00194	0.00028	0.00030	0.00022	0.00030	0.00030
(66)-66-66666	0.00161	0.00019	0.00018	0.00025	0.00015	0.00019
(55)-55-55555	0.00168	0.00031	0.00041	0.00034	0.00082	0.00034

UnsortedCallDB + ArrayList + 16K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.01239	0.00620	0.00506	0.00561	0.00460	0.00561
(66)-05-07765	0.00204	0.00610	0.00185	0.00089	0.00111	0.00185
(60)-12-15378	0.00257	0.00153	0.00090	0.00091	0.00080	0.00091
(48)-16-73476	0.00638	0.00095	0.00083	0.00088	0.00095	0.00095
(38)-25-92205	0.00851	0.00102	0.00098	0.00117	0.00143	0.00117
(99)-99-99999	0.00668	0.00079	0.00094	0.00096	0.00090	0.00094
(88)-88-88888	0.00624	0.00081	0.00123	0.00205	0.00083	0.00123
(77)-77-77777	0.00749	0.00235	0.00080	0.00149	0.00098	0.00149
(66)-66-66666	0.00794	0.00093	0.00097	0.00250	0.00065	0.00097
(55)-55-55555	0.00657	0.00091	0.00101	0.00096	0.00094	0.00096

UnsortedCallDB + ArrayList + 64K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.02295	0.01528	0.00357	0.00250	0.00229	0.00357
(66)-05-07765	0.01253	0.00226	0.00238	0.00247	0.00262	0.00247
(60)-12-15378	0.01278	0.00257	0.00357	0.00253	0.00272	0.00272
(48)-16-73476	0.01192	0.00285	0.00372	0.00465	0.00258	0.00372

(38)-25-92205	0.03414	0.00259	0.00395	0.00276	0.00331	0.00331
(99)-99-99999	0.17206	0.00262	0.00264	0.00282	0.00373	0.00282
(88)-88-88888	0.01281	0.00338	0.04217	0.00185	0.00355	0.00355
(77)-77-77777	0.03228	0.00268	0.00251	0.00964	0.00244	0.00268
(66)-66-66666	0.02602	0.00215	0.00263	0.00490	0.00805	0.00490
(55)-55-55555	0.01240	0.00186	0.00251	0.01774	0.00341	0.00341

UnsortedCallDB + ArrayList + 256k

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.21989	0.04627	0.02343	0.00672	0.00619	0.02343
(66)-05-07765	0.04473	0.00807	0.00601	0.00606	0.00657	0.00657
(60)-12-15378	0.02510	0.00651	0.00617	0.00689	0.00601	0.00651
(48)-16-73476	0.02289	0.00829	0.00582	0.00617	0.02112	0.00829
(38)-25-92205	0.01700	0.00592	0.00450	0.00622	0.02616	0.00622
(99)-99-99999	0.01388	0.00680	0.00692	0.00640	0.00631	0.00680
(88)-88-88888	0.01785	0.02659	0.00669	0.01422	0.00559	0.01422
(77)-77-77777	0.04036	0.00503	0.01340	0.00979	0.01308	0.01308
(66)-66-66666	0.02763	0.00609	0.00641	0.00541	0.04890	0.00641

(55)-55-55555 0.04674 0.00624 0.01040 0.01006 0.00785 0.01006

UnsortedCallDB + LinkedList + 4k

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.00694	0.00270	0.00165	0.00140	0.00083	0.00165
(66)-05-07765	0.00289	0.00149	0.00135	0.00131	0.00223	0.00149
(60)-12-15378	0.01134	0.00152	0.00214	0.00271	0.00136	0.00214
(48)-16-73476	0.00250	0.00210	0.00153	0.00087	0.00242	0.00210
(38)-25-92205	0.00242	0.00079	0.00113	0.00084	0.00123	0.00113
(99)-99-99999	0.00192	0.00057	0.00081	0.00077	0.00168	0.00081
(88)-88-88888	0.00131	0.00078	0.00074	0.00139	0.00110	0.00110
(77)-77-77777	0.00189	0.00057	0.00078	0.00070	0.00111	0.00078
(66)-66-66666	0.00168	0.00079	0.00066	0.00070	0.00072	0.00072
(55)-55-55555	0.00231	0.00501	0.00109	0.00062	0.00076	0.00109

UnsortedCallDB + LinkedList + 16K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.01467	0.00542	0.00517	0.00440	0.00356	0.00517

(66)-05-07765	0.00797	0.00423	0.00284	0.00485	0.00202	0.00423
(60)-12-15378	0.00763	0.00281	0.00270	0.00279	0.00270	0.00279
(48)-16-73476	0.05396	0.00291	0.00283	0.00540	0.00756	0.00540
(38)-25-92205	0.00658	0.00348	0.00796	0.00294	0.00263	0.00348
(99)-99-99999	0.00716	0.00290	0.00279	0.00372	0.00289	0.00290
(88)-88-88888	0.00808	0.00785	0.00760	0.00294	0.00284	0.00760
(77)-77-77777	0.00766	0.00291	0.00280	0.00248	0.00293	0.00291
(66)-66-66666	0.00677	0.00383	0.00396	0.00239	0.00754	0.00396
(55)-55-55555	0.00797	0.00263	0.00351	0.00785	0.00296	0.00351

UnsortedCallDB + LinkedList + 64K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.02708	0.00726	0.01025	0.00434	0.01284	0.01025
(66)-05-07765	0.01592	0.00499	0.00468	0.00494	0.03494	0.00499
(60)-12-15378	0.01625	0.00539	0.00591	0.00553	0.01487	0.00591
(48)-16-73476	0.01440	0.00466	0.00653	0.01516	0.00557	0.00653
(38)-25-92205	0.01632	0.00472	0.00470	0.00381	0.00589	0.00472
(99)-99-99999	0.01569	0.00418	0.00554	0.00486	0.00484	0.00486

(88)-88-88888	0.01559	0.00541	0.00593	0.00439	0.00930	0.00593
(77)-77-77777	0.01053	0.00664	0.00573	0.01460	0.00414	0.00664
(66)-66-66666	0.01528	0.00443	0.00491	0.01117	0.01535	0.01117
(55)-55-55555	0.01737	0.00639	0.00608	0.01328	0.00507	0.00639

UnsortedCallDB + LinkedList + 256K

Number	Run 1	Run 2	Run 3	Run 4	Run 5	Median
(73)-14-91449	0.02360	0.02103	0.02216	0.02604	0.03894	0.02360
(66)-05-07765	0.03414	0.02096	0.02317	0.03350	0.02097	0.02317
(60)-12-15378	0.05797	0.02239	0.03636	0.02112	0.02203	0.02239
(48)-16-73476	0.05916	0.02246	0.01951	0.01494	0.06973	0.02246
(38)-25-92205	0.03884	0.02162	0.02280	0.02387	0.02237	0.02280
(99)-99-99999	0.05922	0.02040	0.02135	0.02101	0.02187	0.02135
(88)-88-88888	0.07773	0.02255	0.02322	0.02102	0.02112	0.02255
(77)-77-77777	0.03428	0.01970	0.02160	0.02783	0.02735	0.02735
(66)-66-66666	0.03704	0.02214	0.02186	0.02089	0.02078	0.02186
(55)-55-55555	0.05693	0.01827	0.04243	0.01754	0.02237	0.02237